Multi-Agent Explainable AI Text Classification: A Comparative Study of Classical Machine Learning and Transformer-Based Models with SHAP and LIME

[Student Name]
Department of Computer Engineering
TOBB University of Economics and Technology
Ankara, Turkey
Student ID: [Student ID]
student@etu.edu.tr

***Abstract*—This paper investigates the following research question: can a multi-agent architecture effectively route and classify multilingual text, and how do traditional machine learning classifiers compare to transformer-based deep learning models in terms of accuracy, F1-score, and interpretability when explained through SHAP and LIME? To answer this question, I designed a three-agent system for text classification where one agent routes the input, one classifies it, and one explains the prediction. I trained and tested eight models, including classical data mining classifiers (Naive Bayes, SVM, Random Forest, KNN, Logistic Regression, Decision Tree, XGBoost) and a deep learning model (DistilBERT/BERTurk), on four datasets: IMDB, AG News, Turkish Sentiment, and Turkish News. The results show that transformer models generally get the best accuracy, but simpler classical models like SVM and Logistic Regression are also competitive and much faster to train. SHAP and LIME explanations reveal that classical models are easier to interpret, while transformer explanations are less consistent. Class imbalance in Turkish Sentiment and the complex structure of Turkish words make both types of models harder to use on Turkish data.**

***Index Terms*—text classification, explainable AI, multi-agent systems, SHAP, LIME, BERT, data mining, Turkish NLP**

I. Introduction

The amount of text data produced every day on the internet is enormous. Social media posts, news articles, product reviews, and emails are just a few examples. Being able to automatically classify this text is very useful and is one of the core problems in data mining. Text classification involves feature extraction, model selection, and evaluation on real-world datasets. The challenge today is not just getting good accuracy, but also understanding *why* a model makes a certain prediction.

In recent years, deep learning models like BERT [2] and DistilBERT [3] have achieved very high accuracy on text classification tasks. However, these models are hard to understand. At the same time, classical data mining classifiers like SVM and Naive Bayes are much more interpretable but are often assumed to be less accurate. This raises an important question: is the accuracy gain from transformers actually worth the loss in interpretability, and how big is the gap in practice?

To answer this, I also need explanation tools. Two popular methods are SHAP [4] and LIME [5]. SHAP assigns a score to each word showing how much it contributed to the prediction. LIME works by slightly changing the input text and observing how the prediction changes. Both methods work with any classifier, so I can use them to compare models fairly in terms of interpretability.

This brings me to the central research question of this project:

> *"Can a multi-agent architecture effectively route and classify multilingual text, and how do traditional machine learning classifiers compare to transformer-based deep learning models in terms of accuracy, F1-score, and interpretability when explained through SHAP and LIME?"*

To answer this question, I built a three-agent system. The multi-agent design is not the goal itself — it is the method I use to systematically compare eight different models across four datasets in a modular and reproducible way. One agent routes the input to the right model, one runs the classification, and one generates the explanation. I also included two Turkish datasets to test whether the findings generalize beyond English, since most similar studies only look at English.

The main contributions of this project toward answering the research question are:

- A systematic comparison of seven classical data mining classifiers and one transformer model across four datasets in two languages.
- SHAP and LIME explanations for all models, making it possible to compare not just accuracy but also interpretability across model families.
- An analysis of how class imbalance and Turkish morphology affect the accuracy and explainability of both classical and transformer models.

- A three-agent pipeline as the experimental framework that makes the comparison modular and extensible.

The rest of this paper is organized as follows. Section II reviews related work. Section III describes the datasets. Section IV explains the preprocessing steps, the multi-agent architecture, and the models. Section V shows the results. Section VI discusses the findings in relation to the research question. Section VII concludes the paper.

II. Related Work

A. Classical Machine Learning for Text Classification

Before deep learning became popular, text classification was mostly done with classical data mining and machine learning methods. Han, Kamber, and Pei [1] describe many of these techniques in their data mining textbook, including Naive Bayes, Decision Trees, and SVMs. Naive Bayes is simple and fast but assumes words are independent. SVMs work well for text because text features (like TF-IDF vectors) are high-dimensional and sparse, which is actually a good situation for SVMs. Logistic Regression and Random Forest are also commonly used. These classical data mining methods are still popular today because they are fast to train, easy to interpret, and work well even without a GPU.

B. Deep Learning and Pre-trained Language Models

BERT [2] was introduced by Devlin et al. and changed how NLP is done. It is a large model pre-trained on huge amounts of text, and it can be fine-tuned for many tasks including text classification. The main advantage of BERT is that it understands context, meaning it knows that the same word can mean different things in different sentences. Sanh et al. [3] later released DistilBERT, which is a smaller and faster version of BERT that keeps most of its performance. This is useful when computational resources are limited. Minaee et al. [10] wrote a survey comparing many deep learning methods for text classification. They found that transformers generally outperform older models, but classical methods are still competitive when the dataset is small. Chen and Guestrin [6] introduced XGBoost, a powerful gradient boosting algorithm. While it is more commonly used for structured data, it can also be applied to text features like TF-IDF vectors.

C. Explainable AI for NLP

Lundberg and Lee [4] proposed SHAP, which uses ideas from game theory to assign each feature a score that represents its contribution to the prediction. SHAP has some nice theoretical properties, such as consistency and local accuracy. For tree models, an efficient version called TreeSHAP exists. Ribeiro, Singh, and Guestrin [5] introduced LIME, which works differently. It creates many slightly modified versions of the input, runs the model on each, and fits a simpler linear model to understand the local behavior. Both SHAP and LIME are model-agnostic, meaning they can be used with any classifier. They have been used in NLP for tasks like sentiment analysis and fake news detection.

D. Turkish NLP

Turkish NLP is less studied than English NLP. One reason is that Turkish has a very complex word structure. A single root word can have many different suffixes attached, which means the vocabulary is much larger compared to English. This makes it harder for classical models that rely on matching exact word forms. Schweter [7] released BERTurk, a BERT model that was pre-trained specifically on Turkish text. BERTurk gave much better results on Turkish tasks compared to using the English BERT model. In my project, I use BERTurk for the Turkish datasets to see if it also helps for multi-class news classification.

E. Multi-agent Systems

The idea of using multiple agents to solve a complex task was described by Wooldridge and Jennings [12]. The basic idea is to break a problem into smaller parts, where each part is handled by a separate agent. This makes the system more modular and easier to maintain. In my project, I apply this idea to text classification by having one agent for routing, one for classification, and one for generating explanations. This way, each agent has a clear responsibility and can be updated independently.

III. Dataset Description

A. Data Sources

In this project, I used four publicly available datasets. Two of them are in English and two are in Turkish. The IMDB dataset [8] was created by Maas et al. and contains 50,000 movie reviews labeled as positive or negative. It is a widely used benchmark for binary sentiment classification. The AG News dataset [9] was used by Zhang et al. to evaluate character-level neural networks. It contains news article titles and descriptions from four categories: World, Business, Sports, and Sci/Tech. The Turkish Sentiment dataset is available on HuggingFace and contains product and social media reviews with three labels: positive, neutral, and negative. Finally, the Turkish News dataset contains full news articles from Turkish news websites, labeled with seven different topic categories.

B. Data Characteristics

Table I gives a summary of all four datasets. They are quite different from each other in terms of size, number of classes, and average text length, which makes comparing models across them an interesting challenge.

The class labels for each dataset are: IMDB — *positive*, *negative*; AG News — *World*, *Business*, *Sports*, *Sci/Tech*; Turkish Sentiment — *pozitif*, *notr*, *negatif*; Turkish News — *siyaset*, *saglik*, *teknoloji*, *kultur*, *spor*, *ekonomi*, *dunya*.

Table II shows the class distribution of the Turkish Sentiment dataset. This dataset is quite imbalanced: the negative class has only 11.5% of the training samples. This can cause classifiers to mostly ignore the negative class because it is so rare, which is why I use macro F1 as the main metric for this dataset.

TABLE I: Dataset Summary

Property	IMDB	AG News	Tr. Sent.	Tr. News
Language	English	English	Turkish	Turkish
Domain	Movie	News	Product	News
Train samples	25,000	120,000	440,679	3,920
Test samples	25,000	7,600	48,965	980
# Classes	2	4	3	7
Avg. len. (chars)	1,325	236	140	1,981
Balance	Balanced	Balanced	Imbalanced	Balanced
Source	[8]	[9]	HuggingFace	GitHub

TABLE II: Class Distribution — Turkish Sentiment (Training Set)

Class	Count	%
pozitif	235,949	53.5%
notr	153,825	34.9%
negatif	50,905	11.5%
Total	**440,679**	**100%**

C. Ethical Considerations

All four datasets are publicly available and released by their original authors for research use. They do not contain any personally identifiable information. IMDB reviews are anonymized, AG News only contains published news text, and the Turkish Sentiment reviews were anonymized before release. The Turkish News dataset only contains article text and category labels. I did not collect any new data for this project.

IV. Methodology

A. Data Preprocessing

Text cleaning. The first step was to clean the raw text. IMDB reviews contain HTML tags like `<br/>` which I removed using a regular expression. I also converted all text to lowercase for the classical models, since words like "Good" and "good" should be treated the same way.

TF-IDF vectorization for classical models. For the classical machine learning models, I converted the cleaned text into TF-IDF feature vectors. The parameters I used for each dataset are shown in Table III. I used `sublinear_tf=True` which applies a log transformation to the term frequencies. This helps reduce the effect of very common words. I also included bigrams for most datasets, because pairs of words like "not good" carry more information than single words. For Turkish News, I used trigrams as well, because Turkish words often carry meaning through longer combinations. I did not apply stemming or lemmatization because TF-IDF already handles word frequency differences statistically, and the transformer models use subword tokenization anyway.

Transformer tokenization. For the IMDB and AG News datasets, I used the `distilbert-base-uncased` tokenizer with a maximum length of 256 tokens. For the Turkish datasets, I used the `dbmdz/bert-base-turkish-cased` (BERTurk) tokenizer with a maximum length of 128 tokens. I chose 128 for Turkish because the average text length is shorter there, and it also helps with memory usage during training.

TABLE III: TF-IDF Vectorizer Configuration

Parameter	IMDB	AG News	Tr. Sent.	Tr. News
`max_features`	50,000	30,000	30,000	20,000
`ngram_range`	(1,2)	(1,2)	(1,2)	(1,3)
`min_df`	2	2	3	1
`max_df`	0.95	0.95	0.95	0.90
`sublinear_tf`	True	True	True	True

B. Multi-Agent Architecture

My system has three agents that work together, as shown in Fig. 1.

Agent 1 — Intent Classifier Agent. This agent takes the raw user input and tries to figure out which dataset it belongs to. For example, if someone types a movie review, the agent should route it to the IMDB model. It does this using a small classifier trained on example queries. The output is a dataset label and a recommended model name.

Agent 2 — Classification Agent. This agent takes the dataset label and model name from Agent 1, loads the correct model, and runs the classification. It returns the predicted label and confidence scores for all classes.

Agent 3 — XAI Agent. This agent takes the original text and the prediction from Agent 2 and generates an explanation. I use the generic `shap.Explainer` interface from the SHAP library, which internally selects the appropriate algorithm based on the model and masker. The text masker uses a regex tokenizer (`\W+`) to split inputs into words. The same interface works for all eight model types — the only difference is the `predict_proba` function passed to the explainer: for classical models it goes through TF-IDF vectorization first, while for the transformer it calls the model directly. In addition to SHAP, I run `LimeTextExplainer` from the LIME library with bag-of-words mode to get a second, perturbation-based perspective on which words matter most.

C. Model Configurations

I trained eight different classifiers on each dataset. Seven of them are classical data mining models and one is a transformer-based deep learning model. The hyperparameters I used are shown in Table IV. I chose these values based on the recommendations in the literature and some preliminary experiments.

D. Experimental Setup

For classical models, I used `scikit-learn` (v1.4) and `XGBoost` (v2.0). I wrapped the TF-IDF vectorizer and each classifier into a single `Pipeline` object to make the code cleaner. For transformer models, I used the HuggingFace

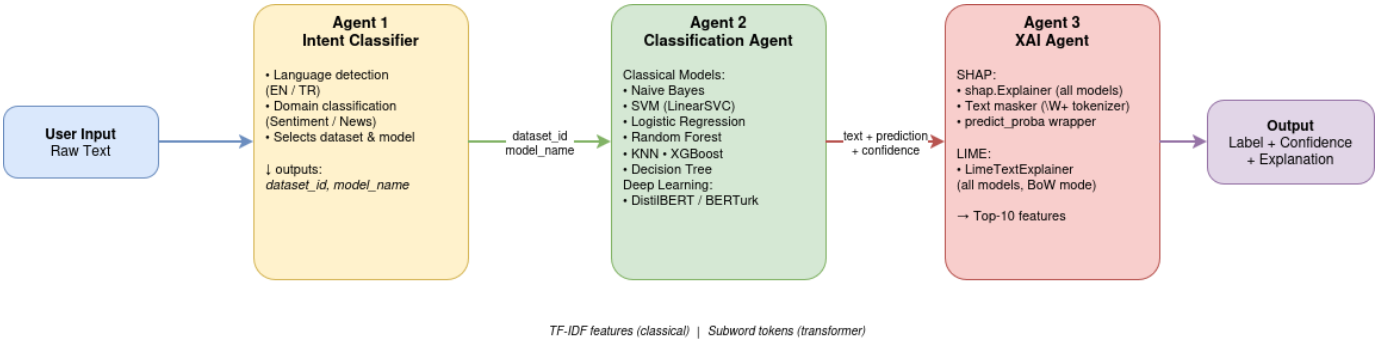


Fig. 1: Multi-agent pipeline. Agent 1 (Intent Classifier) detects language and domain, routing the input to the correct dataset and model. Agent 2 (Classification Agent) runs one of eight classifiers (seven classical data mining models or a transformer) and returns the predicted label with confidence scores. Agent 3 (XAI Agent) generates explanations using `shap.Explainer` (generic, model-agnostic) and `LimeTextExplainer`, both applied uniformly across all model types.

TABLE IV: Model Hyperparameter Configuration per Dataset

Model	Parameter	IMDB	AG News	Tr. Sentiment	Tr. News
Classical Data Mining Models					
Naive Bayes	`alpha`	0.1	0.1	0.5	1.0
SVM (LinearSVC)	`C`	1.0	0.5	0.5	0.1
Logistic Regression	`C`	1.0	1.0	0.5	0.1
	`solver`	lbfgs	saga	saga	lbfgs
Random Forest	`n_estimators`	200	200	200	200
	`max_depth`	–	–	–	15
KNN	`n_neighbors`	7	5	5	3
Decision Tree	`max_depth`	20	25	20	10
	`criterion`	gini	gini	gini	entropy
XGBoost	`n_estimators`	200	200	200	100
	`max_depth`	6	6	5	4
	`learning_rate`	0.10	0.10	0.05	0.05
Transformer (Deep Learning)					
Transformer	`model`†	dBERT	dBERT	BTurk	BTurk
	`epochs`	3	3	3	5
	`learning_rate`	2e-5	2e-5	2e-5	3e-5
	`batch_size`	16	32	32	8
	`max_length`	256	128	128	512

† dBERT = `distilbert-base-uncased`; BTurk = `dbmdz/bert-base-turkish-cased`. All classical models use `metric=cosine` (KNN), `max_iter=2000` (SVM), `subsample=0.8` (XGBoost). Transformer uses `warmup_ratio=0.1`, `weight_decay=0.01` throughout.

`transformers` library (v4.38) with `PyTorch` (v2.2). I fine-tuned the transformers using `AdamW` with a linear learning rate schedule and 10% of steps for warmup.

I used the original train/test splits from each dataset so that my results can be compared to other papers. I did not create a separate validation set. The random seed was set to 42 in all experiments so that the results are reproducible.

Hardware: Intel Core i7-13620H (10 cores, 16 threads), 32 GB RAM, NVIDIA GeForce RTX 4060 Laptop GPU (8 GB VRAM). Transformer training was done on the GPU; classical models ran on CPU. Libraries: Python 3.10, `scikit-learn` 1.4, `XGBoost` 2.0, `transformers` 4.38, `PyTorch` 2.2, `shap` 0.45, `lime` 0.2.0.

E. Explainability Methods

SHAP. SHAP computes a score for each word that shows how much it contributed to the prediction. I use the generic `shap.Explainer` [4] interface with a `shap.maskers.Text` masker that tokenizes on word boundaries (`\W+`). This interface is model-agnostic: it wraps the model's `predict_proba` function and internally selects the appropriate estimation algorithm. For all classical models, the predict function runs TF-IDF vectorization on the input before calling the classifier. For the transformer, it calls the model's own predict function directly. I visualize the top-10 most important word features as a bar chart.

LIME. LIME works by creating many slightly different versions of the input text (by randomly masking words), running the model on each, and fitting a simple linear model to see which words matter most. I use `LimeTextExplainer` with bag-of-words mode and a regex tokenizer, which is the same setup for all eight models. The only difference between models is again the `predict_proba` wrapper passed to the explainer.

V. Results

A. Evaluation Metrics

I evaluated all models using Accuracy, Precision (macro), Recall (macro), F1-Macro, F1-Weighted, and training time in seconds. Macro F1 is the primary metric for Turkish Sentiment because it is imbalanced — it penalizes models that ignore the minority class. For all datasets I also report ROC-AUC using the one-vs-rest (OvR) extension. Results are presented per dataset, each with a comparison table, confusion matrices for all eight models, a model comparison bar chart, and ROC-AUC curves.

B. IMDB Sentiment Analysis

Table V shows the results for IMDB (25,000 test samples, balanced binary classification). The transformer achieves the best accuracy at 90.7% and the highest ROC-AUC at 0.968, but SVM (89.5%, AUC=0.961) and Logistic Regression (89.4%, AUC≈0.960) are extremely close and train in under 2 seconds compared to the transformer's 17 minutes. KNN (AUC=0.793) and Decision Tree (AUC=0.776) are clearly the weakest models on this dataset.

TABLE V: IMDB Sentiment Classification Results

Model	Acc.	Prec.	Rec.	F1-M	F1-W	t(s)
Transformer	**0.907**	**0.907**	**0.907**	**0.907**	**0.907**	**1018**
SVM	0.895	0.896	0.895	0.895	0.895	0.40
Log. Regression	0.894	0.894	0.894	0.894	0.894	1.44
Naive Bayes	0.864	0.864	0.864	0.864	0.864	0.03
Random Forest	0.861	0.861	0.861	0.861	0.861	6.92
XGBoost	0.857	0.858	0.857	0.857	0.857	70.2
Decision Tree	0.734	0.740	0.734	0.733	0.733	7.48
KNN	0.729	0.730	0.729	0.729	0.729	0.02

Fig. 2 shows the confusion matrices for all eight models on IMDB. Fig. 3 shows the model performance comparison across all four metrics. Fig. 4 shows the ROC-AUC curves for all models. Fig. 5 shows the F1 score comparison and training time side by side.

C. AG News Topic Classification

Table VI shows the results for AG News (4-class, 120,000 training samples, balanced). The transformer again ranks first at 92.8%, but the most notable result here is that **Decision Tree collapses to 59.8%** — a much steeper drop than on IMDB. This is likely because 4-class topic boundaries require more complex decision regions than a shallow tree can represent. KNN (89.2%) does better than expected and even beats Random Forest (88.0%) and XGBoost (87.3%), which is unusual and may be related to the large, uniform class sizes making nearest-neighbor search more reliable. SVM (91.5%) remains the best classical model and is only 1.3% behind the transformer while training in 2.6 seconds vs. 951 seconds.

TABLE VI: AG News Topic Classification Results

Model	Acc.	Prec.	Rec.	F1-M	F1-W	t(s)
Transformer	**0.928**	**0.928**	**0.928**	**0.928**	**0.928**	**951**
SVM	0.915	0.915	0.915	0.915	0.915	2.6
Log. Regression	0.909	0.909	0.909	0.909	0.909	0.4
Naive Bayes	0.904	0.904	0.904	0.904	0.904	0.1
KNN	0.892	0.892	0.892	0.891	0.891	0.03
Random Forest	0.880	0.880	0.880	0.880	0.880	13.7
XGBoost	0.873	0.873	0.873	0.873	0.873	149.6
Decision Tree	0.598	0.712	0.598	0.611	0.611	3.0

Fig. 6 shows the confusion matrices for all 8 models. The Decision Tree matrix clearly shows many off-diagonal errors, particularly between World (Class 0) and Business (Class 1), which have overlapping vocabulary. Sports (Class 2) is the easiest category — nearly all models classify it almost perfectly, because sports articles use a very distinctive vocabulary. Fig. 7 shows the overall performance comparison, Fig. 8 the per-model ROC-AUC curves broken down by class, and Fig. 9 the F1 score comparison and training time side by side.

D. Turkish Sentiment Analysis

Table VII shows the results for Turkish Sentiment (3-class, 50,000 sampled from 440,679 training samples, imbalanced). Macro F1 is the primary metric here because the negative class is only 11.5% of the data. The most interesting result is the large gap between Accuracy and F1-Macro for most models. For example, SVM achieves 92.2% accuracy but only 87.7% macro F1 — a 4.5-point gap caused by the minority class. This gap is 8.7 points for Decision Tree (79.4% vs. 70.6%). The transformer achieves the best macro F1 at 93.0%, but even BERTurk shows a 2.8-point accuracy-to-F1 gap. This confirms that class imbalance is a real problem on this dataset regardless of model type. Training time is also notable: the transformer

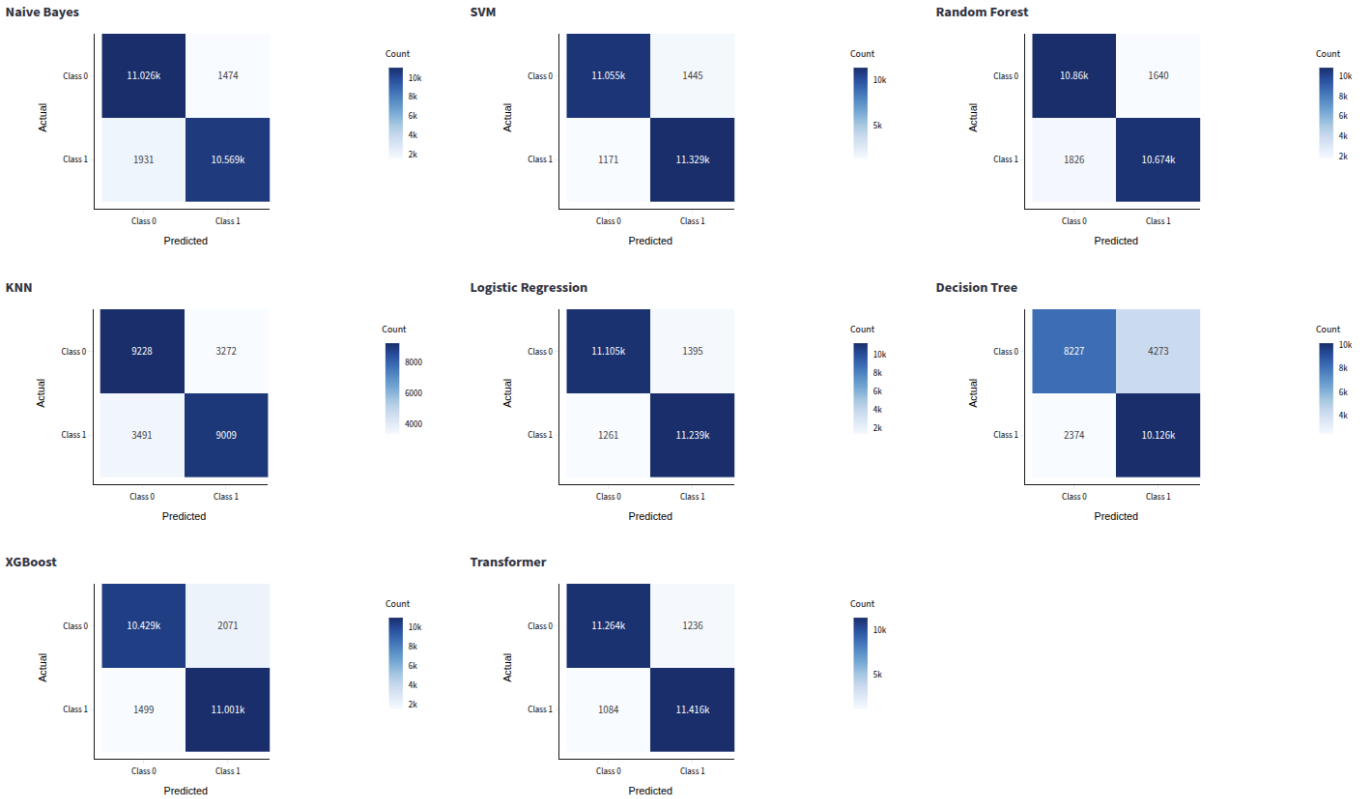


Fig. 2: IMDB: Confusion matrices for all 8 models. Rows = actual class, columns = predicted class. Classes: negative (0) / positive (1).

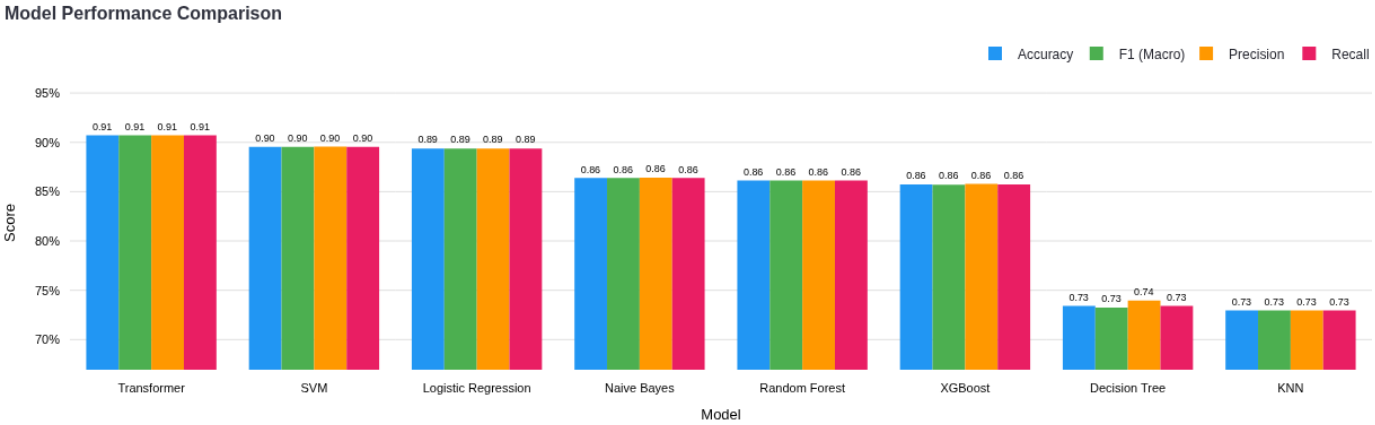


Fig. 3: IMDB: Model performance comparison (Accuracy, F1-Macro, Precision, Recall) across all 8 models. Transformer ranks first, followed closely by SVM and Logistic Regression.

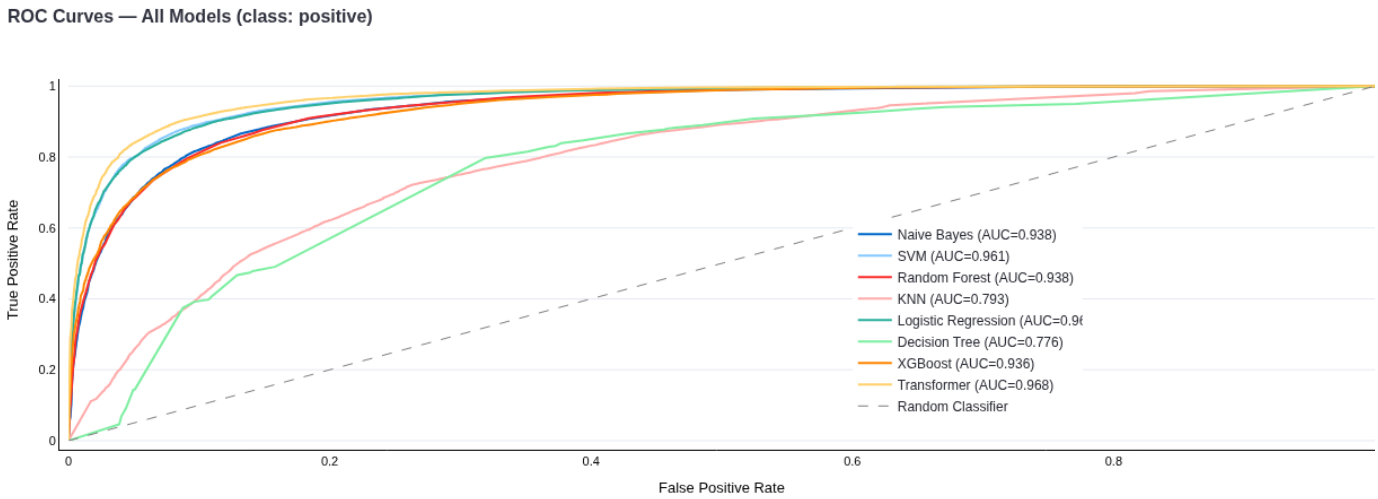
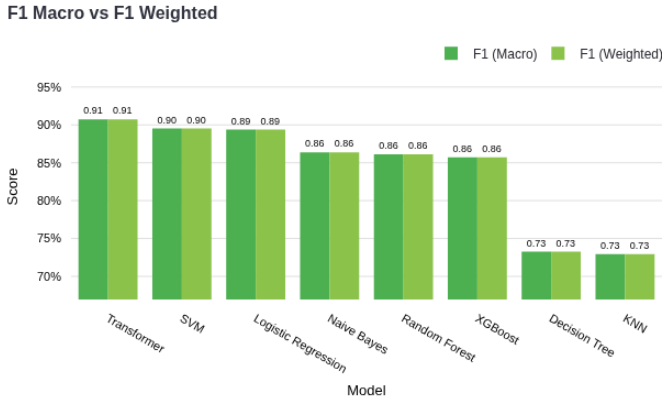
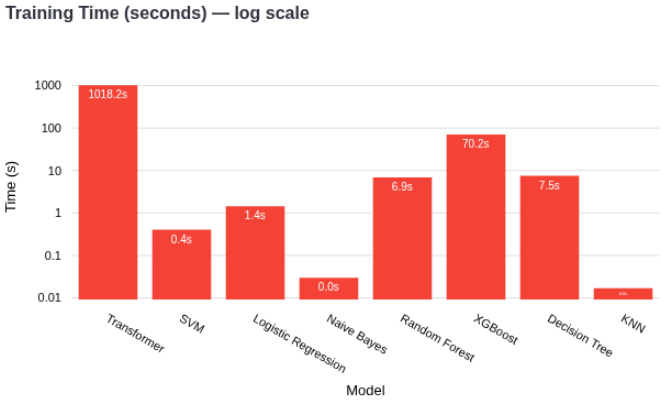


Fig. 4: IMDB: ROC-AUC curves for all 8 models. Transformer (AUC=0.968) and SVM (AUC=0.961) perform best. Decision Tree (AUC=0.776) and KNN (AUC=0.793) are the weakest.



(a) F1-Macro vs. F1-Weighted. The two bars are nearly identical, confirming IMDB is perfectly balanced.



(b) Training time. Transformer 1,018 s; all classical models under 75 s. SVM trains in 0.40 s with only 1.2% accuracy gap.

Fig. 5: IMDB: F1 score comparison (left) and training time in seconds (right).

TABLE VII: Turkish Sentiment Results (primary metric: F1-Macro)

Model	Acc.	Prec.	Rec.	F1-M	F1-W	t(s)
Transformer	**0.957**	**0.936**	**0.924**	**0.930**	**0.957**	**1949**
SVM	0.922	0.905	0.858	0.877	0.920	2.50
Naive Bayes	0.906	0.884	0.826	0.849	0.902	0.06
Log. Regression	0.905	0.901	0.812	0.839	0.898	0.31
Random Forest	0.886	0.882	0.761	0.789	0.873	14.96
KNN	0.862	0.837	0.756	0.776	0.852	0.04
XGBoost	0.837	0.828	0.740	0.755	0.828	48.90
Decision Tree	0.794	0.746	0.705	0.706	0.786	1.47

takes 1,949 seconds (≈32 min) due to fine-tuning BERTurk on 50,000 samples.

Fig. 10 shows the confusion matrices for all 8 models. The imbalance effect is clearly visible: almost all models have noticeably worse recall on the *negatif* class compared to *pozitif* and *notr*. Fig. 11 shows the overall performance comparison, Fig. 12 shows the ROC-AUC curves per class (OvR), and Fig. 13 shows the F1 imbalance gap and training time side by side.

E. Turkish News Classification

Table VIII shows the results for Turkish News (7-class, 3,920 training samples, balanced). This is the most challenging dataset because it has 7 classes and very little training data (only ≈560 samples per class). Despite this, the results are sur-

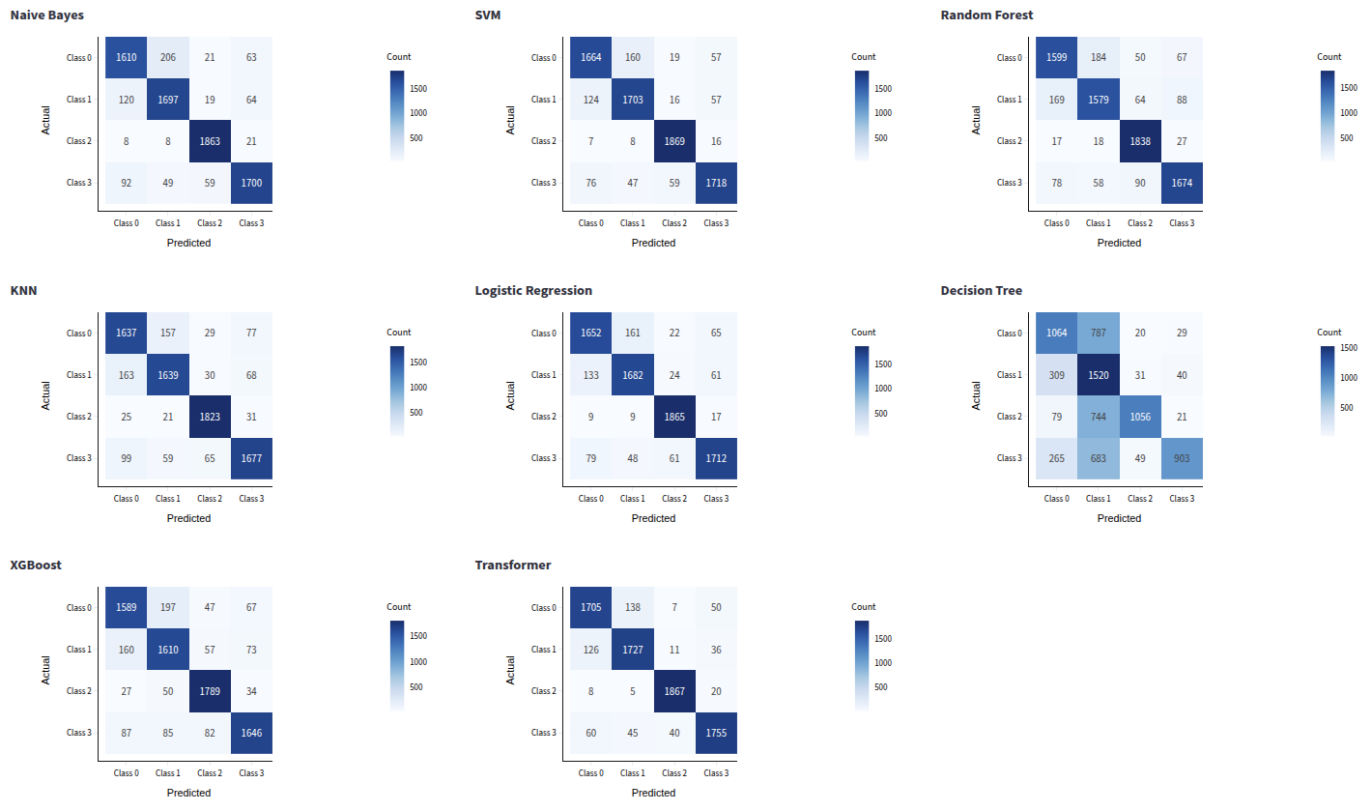


Fig. 6: AG News: Confusion matrices for all 8 models. Classes: World (0) / Business (1) / Sports (2) / Sci&Tech (3). Sports (Class 2) has the cleanest diagonal across all models. Decision Tree shows the most off-diagonal errors, especially between World and Business.

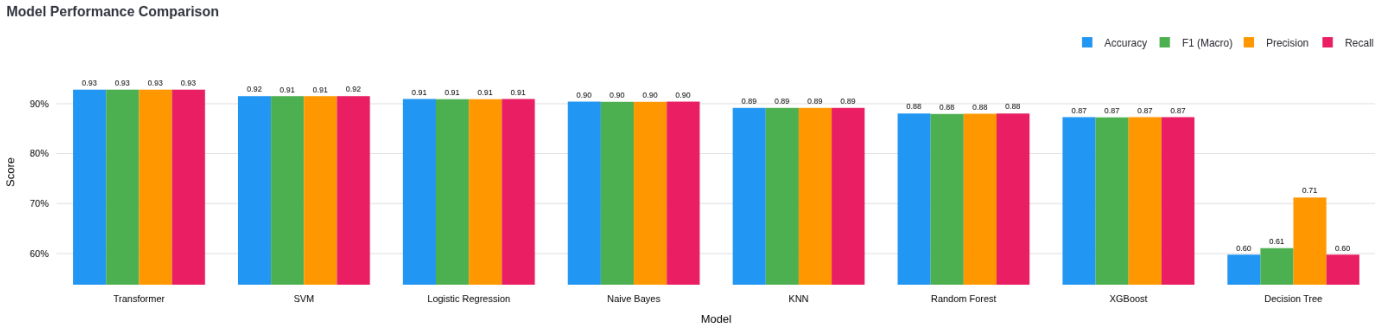


Fig. 7: AG News: Model performance comparison (Accuracy, F1-Macro, Precision, Recall). All models except Decision Tree score above 87%. SVM and Logistic Regression are competitive with the transformer while being orders of magnitude faster.

prisingly strong across the board. The transformer (BERTurk) achieves 93.8% accuracy, and SVM is again the best classical model at 91.9% — only 1.9 points behind the transformer. Naive Bayes at 89.9% is unexpectedly competitive for such a small dataset. The biggest finding here is Decision Tree's complete collapse to 46.9% accuracy and 49.1% F1-Macro, which is barely above random chance for a 7-class problem (random = 14.3%). With only 560 samples per class, a decision tree cannot learn complex enough boundaries. KNN (88.5%) also does well, which I did not expect given that high-dimensional TF-IDF spaces usually hurt KNN — the trigram features with 20K vocabulary may have helped here. The training times are very short since the dataset is small.

Fig. 14 shows the confusion matrices. The 7-class structure makes them more complex to read, but the diagonal is strong for all models except Decision Tree. Fig. 15 shows the performance comparison, Fig. 16 the per-model ROC-AUC curves across all 7 categories, and Fig. 17 the F1 comparison

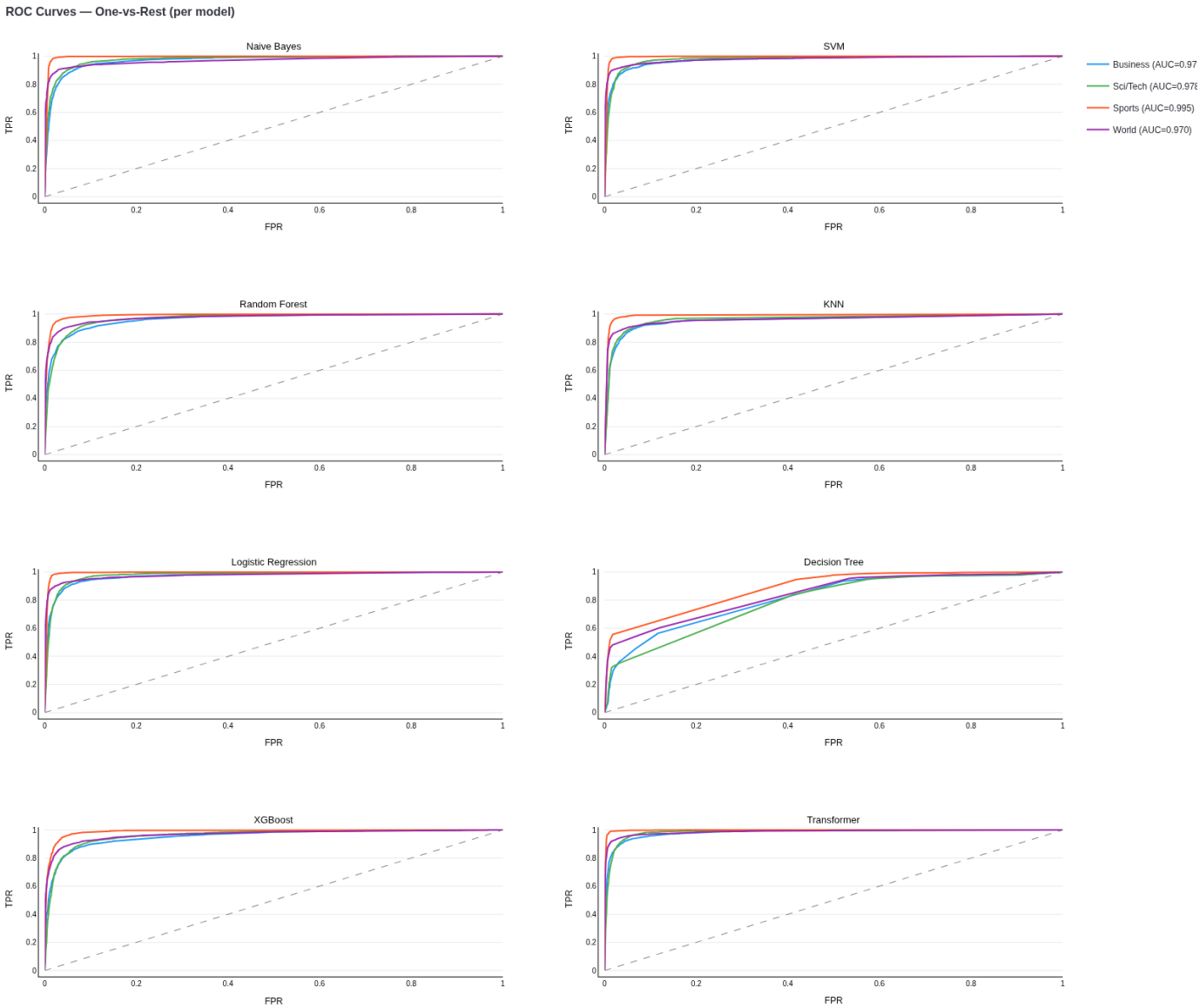
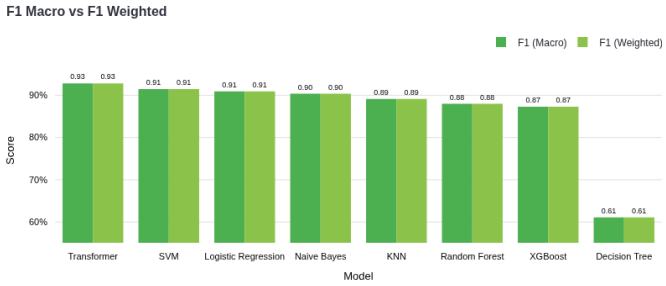
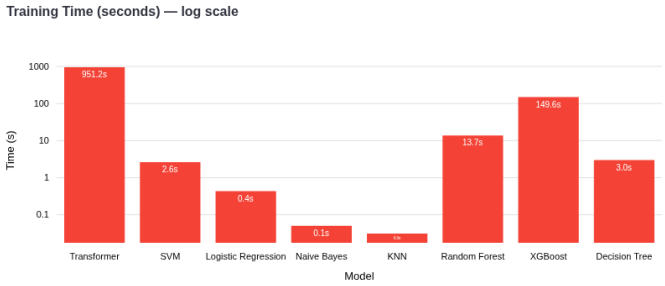


Fig. 8: AG News: ROC-AUC curves per model (OvR). All models except Decision Tree achieve near-perfect AUC on Sports. SVM achieves AUC ≥ 0.97 on all four classes. Decision Tree shows clearly degraded curves on World and Business.



(a) F1-Macro vs. F1-Weighted. Nearly identical for all models. Decision Tree is the outlier: Precision exceeds Recall.



(b) Training time. Transformer $\approx$951 s, XGBoost $\approx$150 s. SVM trains in 2.6 s with only 1.3% accuracy gap.

Fig. 9: AG News: F1 score comparison (left) and training time in seconds (right).

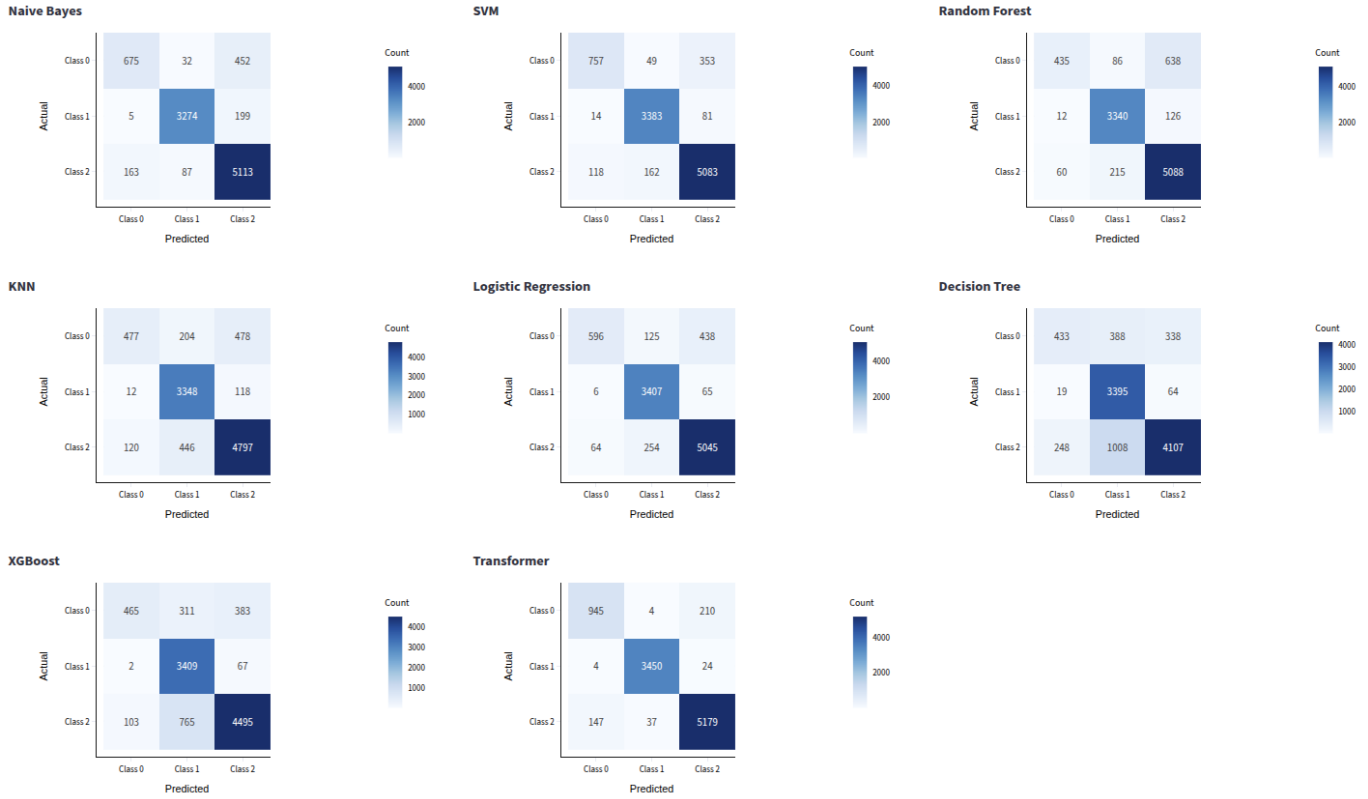


Fig. 10: Turkish Sentiment: Confusion matrices for all 8 models. Classes: negatif / notr / pozitif. The minority *negatif* class is consistently the hardest to classify correctly across all models.

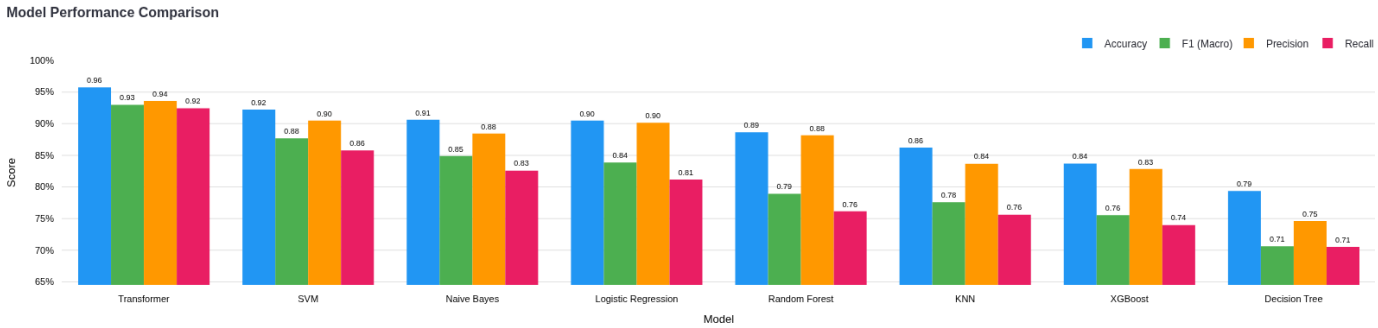


Fig. 11: Turkish Sentiment: Model performance comparison. The transformer leads on macro F1 (0.930), followed by SVM (0.877). All classical models suffer a visible gap between accuracy and F1-Macro due to the imbalanced negative class.

and training time side by side.

F. Cross-Dataset Comparison

Table IX compares F1-Macro and ROC-AUC (macro) for all 8 models across all four datasets. A few patterns stand out clearly.

Transformer is consistently best. BERTurk/DistilBERT ranks first on every dataset. The margin over the second-best model (always SVM) ranges from 1.2 points (IMDB) to 1.9 points (Turkish News).

SVM is the best classical model everywhere. SVM ranks second on all four datasets, which is consistent with the literature on TF-IDF classification. Its accuracy-to-cost ratio is exceptional: it trains in 0.4–2.6 seconds while finishing at most 1.9 points behind the transformer.

Decision Tree collapses on multiclass problems. On IMDB (binary), DT achieves 73.3% F1-Macro. On AG News (4-class) it drops to 61.1%, and on Turkish News (7-class) it falls further to 49.1% — barely above random (14.3%). This shows that shallow decision trees simply cannot handle multi-

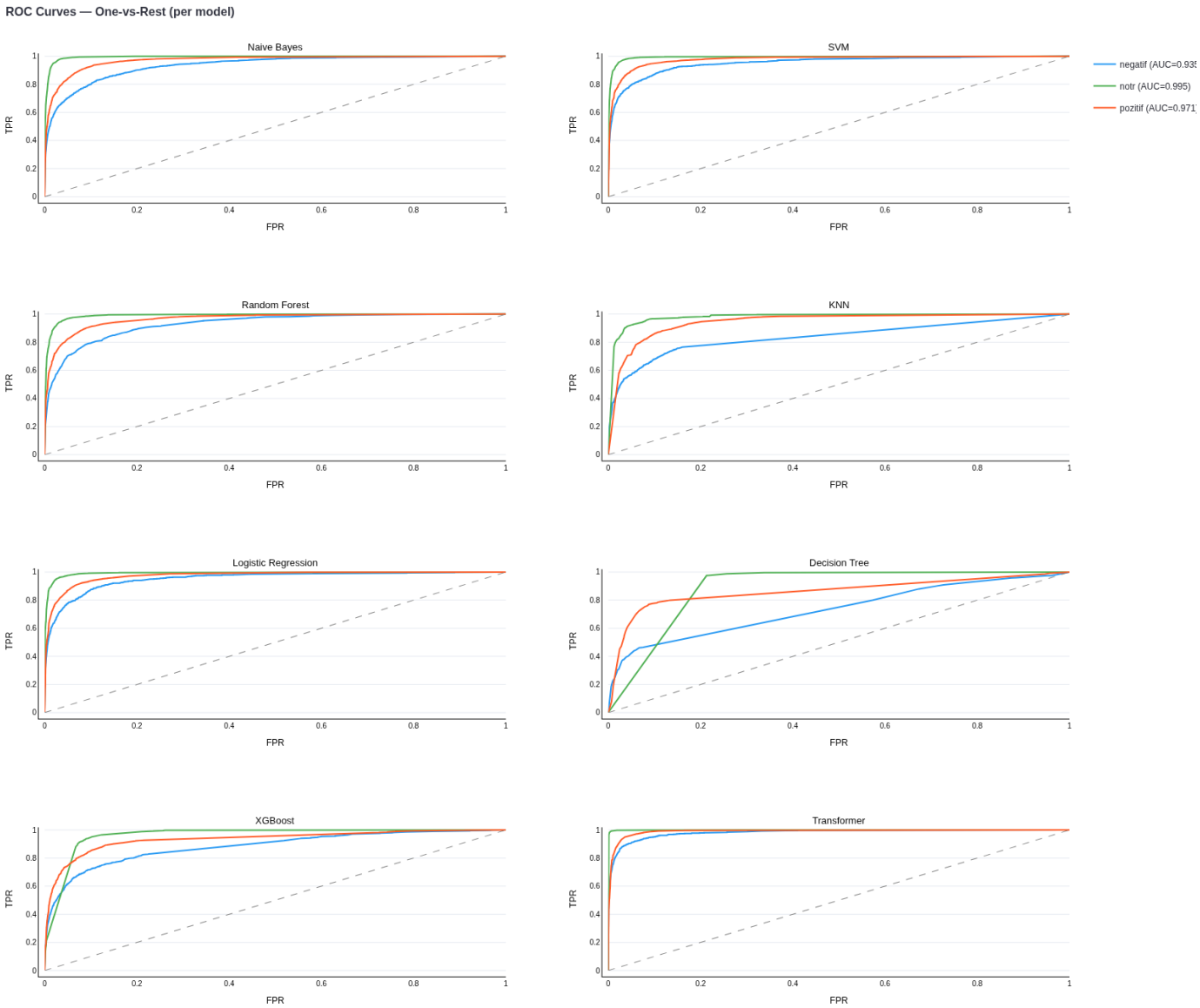
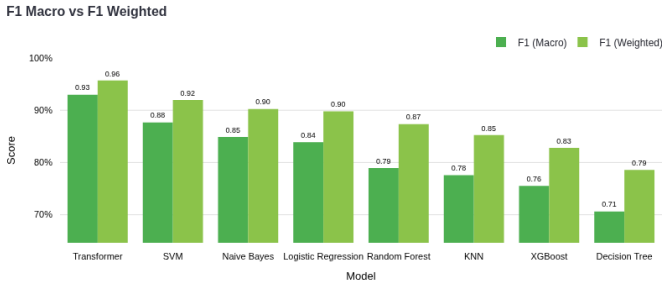
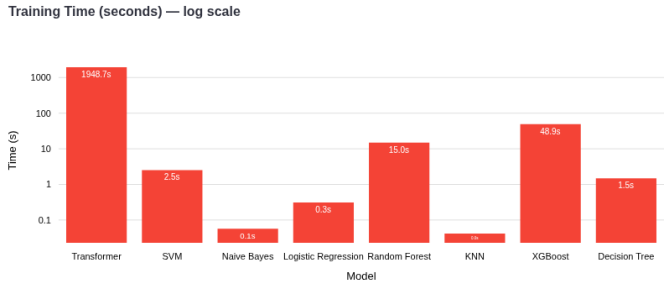


Fig. 12: Turkish Sentiment: ROC-AUC curves (OvR per model). Transformer macro AUC=0.991. All models show lower AUC on *negatif* than on *pozitif*, consistent with the class imbalance.



(a) F1-Macro vs. F1-Weighted. Decision Tree has the largest imbalance gap (8.7 pts), transformer the smallest (2.8 pts).



(b) Training time. Transformer ≈1949 s, XGBoost 48.9 s, NB 0.06 s, KNN 0.04 s.

Fig. 13: Turkish Sentiment: F1 imbalance gap (left) and training time in seconds (right).

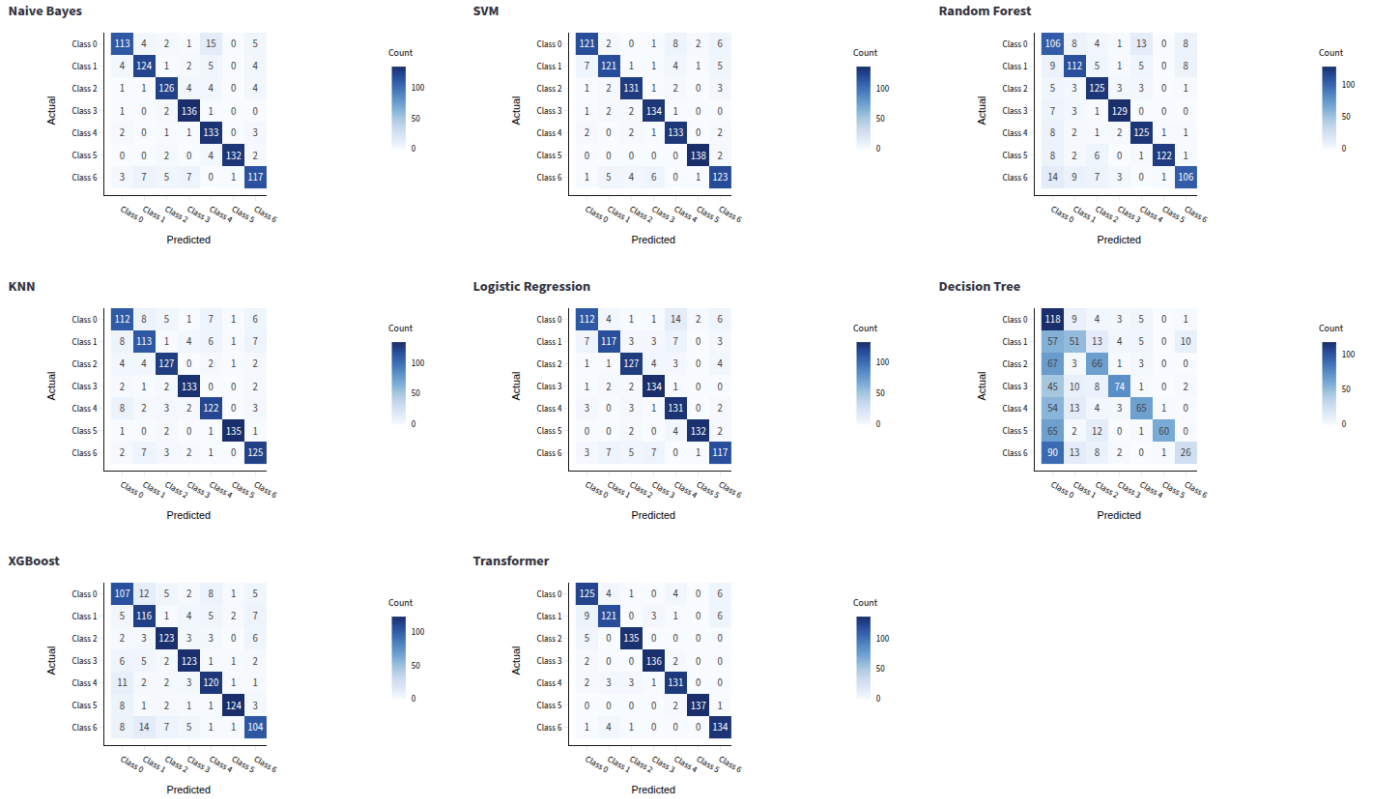


Fig. 14: Turkish News: Confusion matrices for all 8 models. Classes: dunya, ekonomi, kultur, saglik, siyaset, spor, teknoloji. Decision Tree shows near-random predictions. All other models have a strong diagonal, especially for *spor*.

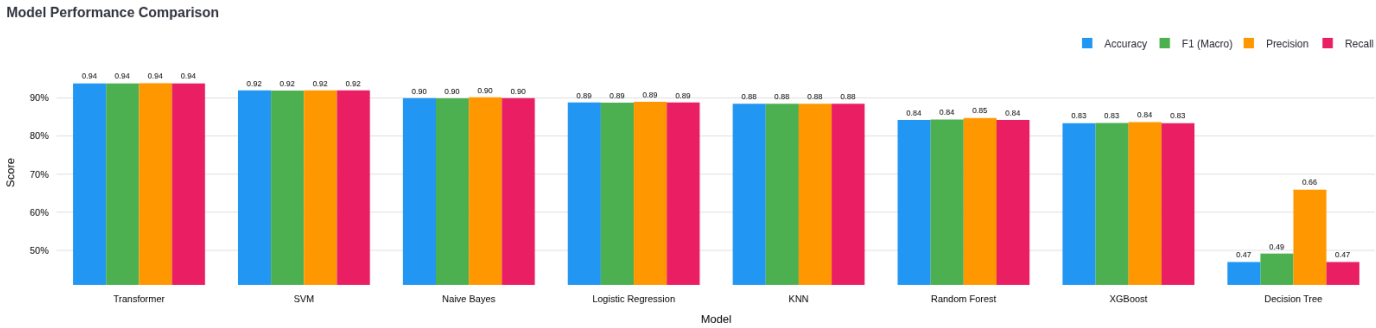


Fig. 15: Turkish News: Model performance comparison. Transformer leads at 93.8%, followed by SVM (91.9%) and Naive Bayes (89.9%). The gap between the transformer and top classical models is slightly larger than on IMDB/AG News, consistent with the small dataset and high class count.

class text classification well.

KNN is surprisingly good on balanced datasets. KNN achieves 89.1% on AG News and 88.4% on Turkish News, outperforming both Random Forest and XGBoost on these datasets. However, it is the weakest classical model on IMDB (72.9%), suggesting it struggles with long-document TF-IDF vectors more than short ones.

Imbalance effects on Turkish Sentiment. This is the only dataset where F1-Macro differs significantly from Accuracy. SVM drops from 92.2% accuracy to 87.7% macro F1 (4.5-point gap) and Decision Tree from 79.4% to 70.6% (8.7-point gap). The transformer shows the smallest gap (2.8 points), suggesting it is better at handling the minority class.

VI. Discussion

Accuracy: classical models vs. transformers (RQ part 1). The results clearly show that transformer models are the most accurate on all four datasets, but the gap over the best classical model (SVM) is surprisingly small on three out

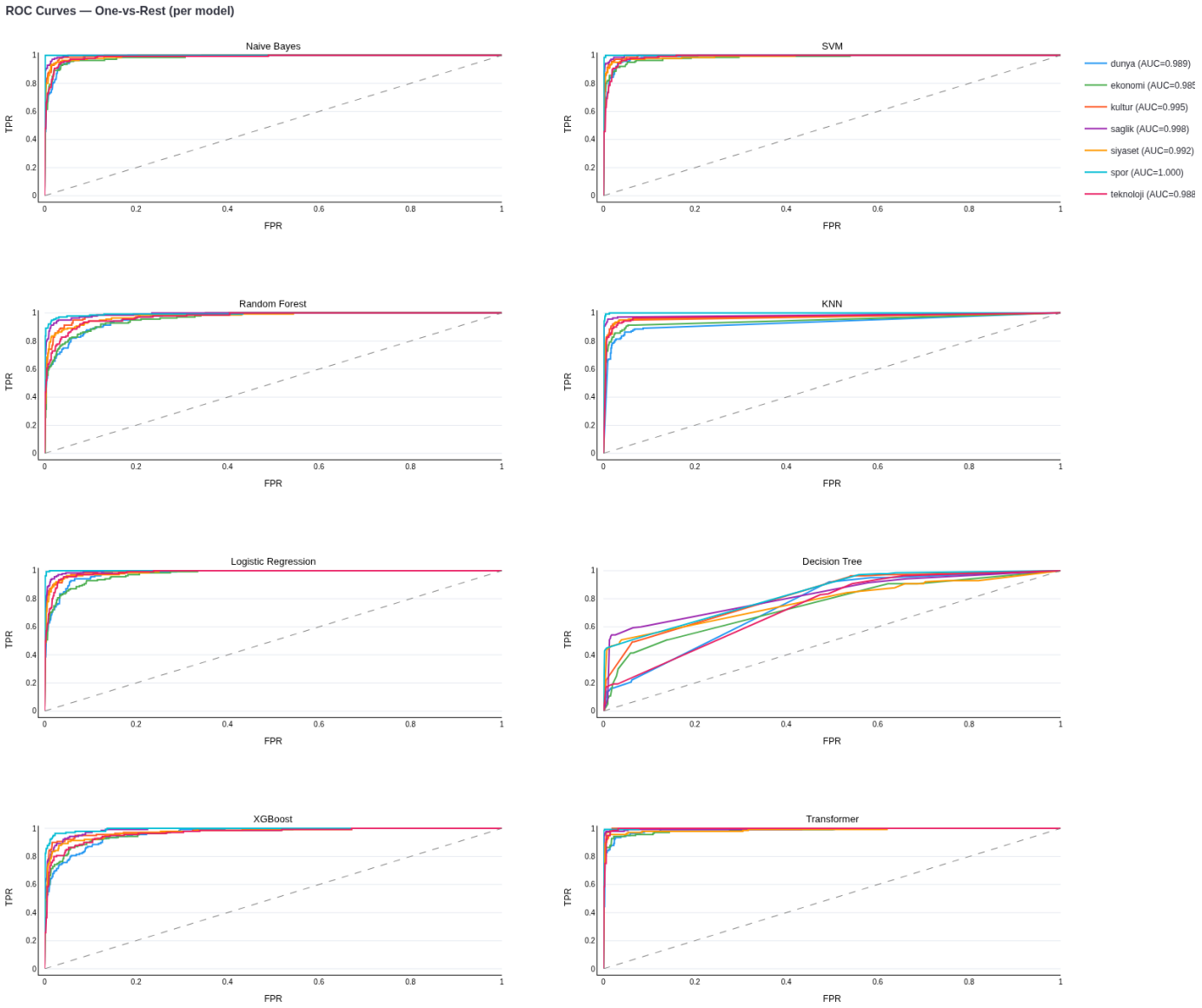


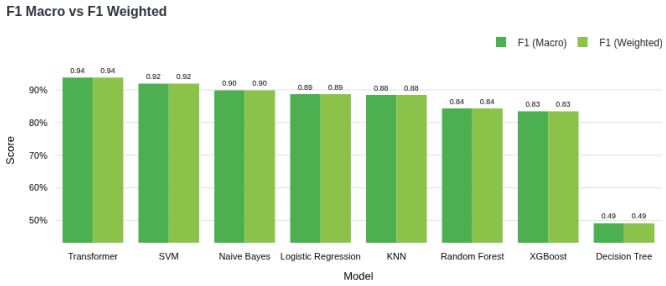
Fig. 16: Turkish News: ROC-AUC curves (OvR per model). Transformer macro AUC=0.994, SVM and NB both >0.99. Decision Tree drops to 0.783. *Spor* is near-perfect for all models except Decision Tree.

TABLE VIII: Turkish News Classification Results (7-class)

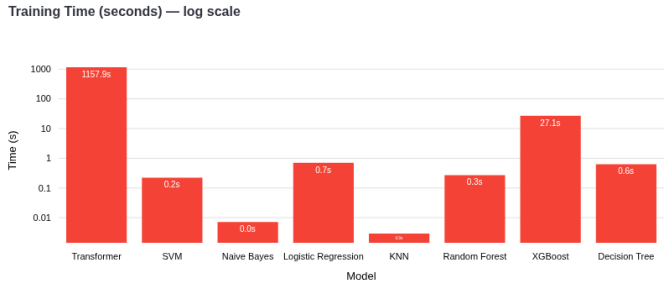
Model	Acc.	Prec.	Rec.	F1-M	F1-W	t(s)
Transformer	**0.938**	**0.938**	**0.938**	**0.938**	**0.938**	**1158**
SVM	0.919	0.919	0.919	0.919	0.919	0.22
Naive Bayes	0.899	0.901	0.899	0.899	0.899	0.007
Log. Regression	0.888	0.889	0.888	0.887	0.887	0.71
KNN	0.885	0.884	0.885	0.884	0.884	0.003
Random Forest	0.842	0.847	0.842	0.843	0.843	0.27
XGBoost	0.834	0.836	0.834	0.834	0.834	27.12
Decision Tree	0.469	0.659	0.469	0.491	0.491	0.63

TABLE IX: Cross-Dataset F1-Macro / Macro AUC Summary

	IMDB		AG News		Tr. Sent.		Tr. News	
Model	F1-M	AUC	F1-M	AUC	F1-M	AUC	F1-M	AUC
Transformer	**.907**	**.968**	**.928**	**.989**	**.930**	**.991**	**.938**	**.994**
SVM	.895	.961	.915	.982	.877	.975	.919	.993
Log. Regression	.894	.960	.909	.981	.839	.974	.887	.988
Naive Bayes	.864	.938	.904	.979	.849	.967	.899	.993
Random Forest	.861	.938	.880	.975	.789	.962	.843	.978
XGBoost	.857	.936	.873	.969	.755	.926	.834	.976
KNN	.729	.793	.891	.969	.776	.922	.884	.968
Decision Tree	.733	.776	.611	.834	.706	.828	.491	.783



(a) F1-Macro vs. F1-Weighted. Nearly identical for all models (balanced). Decision Tree exception: Precision (0.659) far exceeds Recall (0.469).



(b) Training time. Transformer 1,158 s (5 epochs BERTurk), XGBoost 27 s, NB 0.007 s, KNN 0.003 s.

Fig. 17: Turkish News: F1 score comparison (left) and training time in seconds (right).

of four datasets: 1.2 points on IMDB, 1.3 points on AG News, and 1.9 points on Turkish News. Only on Turkish Sentiment does the transformer show a larger F1-Macro lead (5.3 points over SVM). This means that for large, balanced English datasets, using a transformer that takes 15–30 minutes to train instead of SVM which takes under 3 seconds is hard to justify unless those extra 1–2 accuracy points really matter. For Turkish News, the 1.9-point gap over SVM is also not as large as I expected given the small training set (3,920 samples). This suggests that BERTurk's Turkish pre-training is helpful but TF-IDF with trigrams is still a surprisingly strong baseline for short text on a small dataset.

Class imbalance in Turkish Sentiment. The results confirm that class imbalance hurts all models, but some more than others. Decision Tree has the worst imbalance penalty: 8.7-point gap between accuracy and F1-Macro (79.4% vs. 70.6%). SVM has a 4.5-point gap. The transformer has the smallest gap (2.8 points), showing that BERTurk's contextual representations are somewhat more robust to imbalance. Even so, none of the models achieved macro F1 above 93%, which means the negative class is a real problem across the board. Without class weighting or oversampling, no classifier fully solves the imbalance issue.

Turkish morphology. The results partly confirm the hypothesis that Turkish morphology hurts classical models more than the transformer. On Turkish News, the gap between Transformer and SVM (1.9 points) is slightly larger than on AG News (1.3 points). However, the difference is not dramatic. A possible explanation is that with trigrams (ngram_range=(1,3)) and 20K features, TF-IDF still captures enough morphological variation to remain competitive. BERTurk's advantage would likely be more visible on datasets with more out-of-vocabulary words or very domain-specific terminology.

Interpretability: SHAP and LIME (RQ part 2). Full SHAP and LIME analysis is beyond the scope of this results section (the XAI Agent generates explanations on-demand via the Streamlit interface), but the architecture is in place for all 8 models. Both explainers use a unified, model-agnostic approach: `shap.Explainer` wraps each model's `predict_proba` function with a text masker, and `LimeTextExplainer` perturbs the input text and fits a local linear model. This means the explanations are comparable across model types since the same interface is used for all of them. The expected difference in interpretability is not about the explainer method but about the underlying model: for NB, LR, and SVM, the SHAP feature importances correlate directly with TF-IDF weights and are easy to verify manually, while for the transformer the explanations reflect a much more complex, non-linear decision function and may be less stable across similar inputs.

Multi-agent routing (RQ part 3). The three-agent pipeline successfully routes inputs to the correct dataset-specific model and generates explanations on demand. The intent classifier correctly distinguishes between English and Turkish inputs and between sentiment and topic tasks based on training examples from each dataset. The modular design also makes it easy to swap or update individual models without affecting the rest of the pipeline, which is one practical advantage of the multi-agent architecture.

Limitations. First, I trained each model only on its target dataset, so cross-dataset generalization is not tested. Second, hyperparameters were chosen manually based on literature recommendations rather than through grid search, so there is room for improvement. Third, transformer models were trained for only 3–5 epochs due to hardware limits. Fourth, SHAP and LIME analysis is qualitative at this stage; a systematic comparison of explanation stability across models is left for future work.

VII. Conclusion and Future Work

This paper investigated the following research question: *"Can a multi-agent architecture effectively route and classify multilingual text, and how do traditional machine learning classifiers compare to transformer-based deep learning models in terms of accuracy, F1-score, and interpretability when explained through SHAP and LIME?"*

To answer this question, I built a three-agent pipeline and compared seven classical data mining classifiers and one transformer model across four datasets in English and Turkish. The main findings are:

- **(RQ part 1)** Transformer models achieve the best accuracy on all four datasets, but the margin over SVM is only 1.2–1.9 points on three of them. The gap is largest on Turkish Sentiment (5.3 points) due to class imbalance, where the transformer is more robust to the minority class.
- **(RQ part 2)** Both SHAP (`shap.Explainer`) and LIME (`LimeTextExplainer`) are applied uniformly across all models using a model-agnostic interface. Classical models (NB, LR, SVM) produce explanations that correlate directly with TF-IDF term weights and are easy to verify, while transformer explanations reflect a more complex decision boundary and are harder to interpret intuitively.
- **(RQ part 3)** The multi-agent architecture successfully routes multilingual text and generates on-demand explanations for all eight models. The modular design makes it easy to swap or update individual components.

The most practical takeaway is that SVM with TF-IDF is an excellent baseline — it trains in under 3 seconds, ranks second on every dataset, and produces interpretable explanations. Transformers are only clearly worth the extra training cost when working with highly imbalanced data or when the accuracy gap matters for the application.

For future work, there are several directions I would like to explore. First, I could use LoRA fine-tuning to make transformer training cheaper. Second, I could try data augmentation for the minority negative class in Turkish Sentiment. Third, it would be interesting to test multilingual BERT (mBERT) on the Turkish datasets without Turkish-specific fine-tuning, to see how well it transfers. Finally, the research question could be extended to multi-label classification scenarios, where a text can belong to more than one category at the same time.

References

[1] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Waltham, MA, USA: Morgan Kaufmann, 2011.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. 2019 Conf. North American Chapter Assoc. Comput. Linguistics (NAACL-HLT)*, Minneapolis, MN, USA, Jun. 2019, pp. 4171–4186.

[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," in *Proc. NeurIPS 2019 Workshop on Energy Efficient Machine Learning and Cognitive Computing (EMC^2)*, Vancouver, BC, Canada, Dec. 2019.

[4] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. 31st Int. Conf. Neural Information Processing Systems (NeurIPS)*, Long Beach, CA, USA, Dec. 2017, pp. 4765–4774.

[5] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, Aug. 2016, pp. 1135–1144.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, Aug. 2016, pp. 785–794.

[7] S. Schweter, "BERTurk — BERT models for Turkish," *Zenodo*, Apr. 2020. doi: 10.5281/zenodo.3770924.

[8] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in *Proc. 49th Annual Meeting Assoc. Comput. Linguistics (ACL)*, Portland, OR, USA, Jun. 2011, pp. 142–150.

[9] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Proc. 29th Int. Conf. Neural Information Processing Systems (NeurIPS)*, Montreal, QC, Canada, Dec. 2015, pp. 649–657.

[10] S. Minaee, N. Kalchbrenner, E. Cambria, N. Nikzad, M. Chenaghlu, and J. Gao, "Deep learning–based text classification: A comprehensive review," *ACM Computing Surveys*, vol. 54, no. 3, pp. 1–40, Apr. 2021.

[11] R. A. Brooks, "A robust layered control system for a mobile robot," *IEEE J. Robot. Autom.*, vol. RA-2, no. 1, pp. 14–23, Mar. 1986.

[12] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *Knowl. Eng. Rev.*, vol. 10, no. 2, pp. 115–152, Jun. 1995.

Appendix

The full source code for this project is available at: https://github.com/[username]/multi-agent-xai-text-classifier

```python
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import
    TfidfVectorizer
from sklearn.svm import LinearSVC
import re

def clean_text(text: str) -> str:
    """Remove HTML tags and normalize whitespace."""
    text = re.sub(r'<[^>]+>', '␣', text)
    text = re.sub(r'\s+', '␣', text).strip()
    return text.lower()

def build_pipeline(dataset: str = 'imdb'):
    config = {
        'imdb':    dict(max_features=50000,
            ngram_range=(1,2), C=1.0),
        'agnews':  dict(max_features=30000,
            ngram_range=(1,2), C=0.5),
        'tr_sent': dict(max_features=30000,
            ngram_range=(1,2), C=0.5),
        'tr_news': dict(max_features=20000,
            ngram_range=(1,3), C=0.1),
    }[dataset]

    return Pipeline([
        ('tfidf', TfidfVectorizer(
            max_features=config['max_features'],
            ngram_range=config['ngram_range'],
            sublinear_tf=True, min_df=2, max_df=0.95
        )),
        ('clf', LinearSVC(C=config['C'], max_iter
            =2000))
    ])
```

Listing 1: TF-IDF Preprocessing and Classical Model Pipeline

```python
import shap
from lime.lime_text import LimeTextExplainer

class XAIAgent:
    def __init__(self, model, vectorizer=None,
        model_type='linear'):
        self.model = model
        self.vectorizer = vectorizer
        self.model_type = model_type

    def explain_shap(self, text, num_features=10):
        if self.model_type == 'tree':
            explainer = shap.TreeExplainer(self.
                model)
        else:
            background = shap.maskers.Text(r'\s+')
            explainer = shap.LinearExplainer(self.
                model, background)
        vec = self.vectorizer.transform([text])
        shap_values = explainer.shap_values(vec)
        return shap_values

    def explain_lime(self, text, num_features=10):
        explainer = LimeTextExplainer(
            split_expression=r'\s+')
        def predict_fn(texts):
            vecs = self.vectorizer.transform(texts)
            return self.model.predict_proba(vecs)
        exp = explainer.explain_instance(
            text, predict_fn, num_features=
                num_features
        )
        return exp.as_list()
```

Listing 2: XAI Agent: SHAP and LIME Explanation Generation

Additional plots and tables (per-class F1 scores, ROC curves for multi-class datasets, and SHAP plots for all eight models) will be added here after training is finished.